trong đó $\mathbf{w} = [w_1, w_2, w_3]^T$ là *vector trọng số* (weight vector) cần tìm. Mối quan hệ $y \approx f(\mathbf{x})$ như trong (3.1) là một mối quan hệ tuyến tính.

Bài toán trên đây là bài toán dự đoán giá trị của đầu ra dựa trên vector đặc trưng đầu vào. Ngoài ra, giá trị của đầu ra có thể nhận rất nhiều giá trị thực dương khác nhau. Vì vậy, đây là một bài toán hồi quy. Mối quan hệ $\hat{y} = \mathbf{x}^T\mathbf{w}$ là một mối quan hệ tuyến tính. Tên gọi *hồi quy tuyến tính* xuất phát từ đây.

3.2. Xây dựng và tối ưu hàm mất mát

Tổng quát, nếu mỗi điểm dữ liệu được mô tả bởi một vector đặc trưng d chiều $\mathbf{x} \in \mathbb{R}^d$, hàm dự đoán đầu ra được viết dưới dạng

$$y = w_1x_1 + w_2x_2 + \cdots + w_dx_d = \mathbf{x}^T\mathbf{w}. \tag{3.2}$$

3.2.1. Sai số dự đoán

Sau khi xây dựng được mô hình dự đoán đầu ra như (3.2), ta cần tìm một phép đánh giá phù hợp với bài toán. Với bài toán hồi quy nói chung, ta mong muốn sự sai khác e giữa đầu ra thực sự y và đầu ra dự đoán $\hat{y}$ là nhỏ nhất:

$$\frac{1}{2}e^2 = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(y - \mathbf{x}^T\mathbf{w})^2. \tag{3.3}$$

Ở đây, bình phương được lấy vì $e = y - \hat{y}$ có thể là một số âm. Việc sai số là nhỏ nhất có thể được mô tả bằng cách lấy trị tuyệt đối $|e| = |y - \hat{y}|$. Tuy nhiên, cách làm này ít được sử dụng vì hàm trị tuyệt đối không khả vi tại gốc toạ độ, không thuận tiện cho việc tối ưu. Hệ số $\frac{1}{2}$ sẽ bị triệt tiêu khi lấy đạo hàm của e theo tham số mô hình $\mathbf{w}$.

3.2.2. Hàm mất mát

Điều tương tự xảy ra với tất cả các cặp dữ liệu $(\mathbf{x}_i, y_i), i = 1, 2, \ldots, N$, với N là số lượng dữ liệu trong tập huấn luyện. Việc tìm mô hình tốt nhất tương đương với việc tìm $\mathbf{w}$ để hàm số sau đạt giá trị nhỏ nhất:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \mathbf{x}_i^T\mathbf{w})^2. \tag{3.4}$$

Hàm số $\mathcal{L}(\mathbf{w})$ chính là hàm mất mát của mô hình hồi quy tuyến tính với tham số $\theta = \mathbf{w}$. Ta luôn mong muốn sự mất mát là nhỏ nhất, điều này có thể đạt được bằng cách tối thiểu hàm mất mát theo $\mathbf{w}$:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\, \mathcal{L}(\mathbf{w}). \tag{3.5}$$

$\mathbf{w}^*$ là nghiệm cần tìm của bài toán.

Trung bình sai số

Trong học máy , hàm mất mát thường là trung bình cộng của sai số tại mỗi điểm. Về mặt toán học, hệ số $\frac{1}{2N}$ không ảnh hưởng tới nghiệm của bài toán. Tuy nhiên, việc lấy trung bình này quan trọng vì số lượng điểm dữ liệu trong tập huấn luyện có thể thay đổi. Việc tính toán mất mát trên từng điểm dữ liệu sẽ hữu ích hơn trong việc đánh giá chất lượng mô hình. Ngoài ra, việc lấy trung bình cũng giúp tránh hiện tượng tràn số khi số lượng điểm dữ liệu lớn.

Trước khi xây dựng nghiệm cho bài toán tối ưu hàm mất mát, ta thấy rằng hàm số này có thể được viết gọn lại dưới dạng ma trận, vector, và chuẩn như sau:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \mathbf{x}_i^T\mathbf{w})^2 = \frac{1}{2N}\left\|\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} - \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}\mathbf{w}\right\|_2^2 = \frac{1}{2N}\|\mathbf{y} - \mathbf{X}^T\mathbf{w}\|_2^2 \quad (3.6)$$

với $\mathbf{y} = [y_1, y_2, \ldots, y_N]^T$, $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N]$. Như vậy $\mathcal{L}(\mathbf{w})$ là một hàm số liên quan tới bình phương của chuẩn ℓ_2.

3.2.3. Nghiệm của hồi quy tuyến tính

Nhận thấy rằng hàm mất mát $\mathcal{L}(\mathbf{w})$ có gradient tại mọi $\mathbf{w}$. Giá trị tối ưu của $\mathbf{w}$ có thể tìm được thông qua việc giải phương trình đạo hàm của $\mathcal{L}(\mathbf{w})$ theo $\mathbf{w}$ bằng không. Gradient của hàm số này tương đối đơn giản:

$$\frac{\nabla\mathcal{L}(\mathbf{w})}{\nabla\mathbf{w}} = \frac{1}{N}\mathbf{X}(\mathbf{X}^T\mathbf{w} - \mathbf{y}) \quad (3.7)$$

Phương trình gradient bằng không:

$$\frac{\nabla\mathcal{L}(\mathbf{w})}{\nabla\mathbf{w}} = \mathbf{0} \Leftrightarrow \mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y} \quad (3.8)$$

Nếu ma trận vuông $\mathbf{X}\mathbf{X}^T$ khả nghịch, phương trình (3.8) có nghiệm duy nhất $\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{y}$.

Nếu ma trận $\mathbf{X}\mathbf{X}^T$ không khả nghịch, phương trình (3.8) vô nghiệm hoặc có vô số nghiệm. Lúc này, một nghiệm đặc biệt của phương trình có thể được xác định dựa vào *giả nghịch đảo*. Người ta chứng minh được rằng với mọi ma trận $\mathbf{X}$, luôn tồn tại duy nhất một giá trị $\mathbf{w}$ có chuẩn ℓ_2 nhỏ nhất giúp tối thiểu $\|\mathbf{X}^T\mathbf{w} - \mathbf{y}\|_F^2$. Cụ thể, $\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger\mathbf{X}\mathbf{y}$ trong đó $(\mathbf{X}\mathbf{X}^T)^\dagger$ là giả nghịch đảo của $\mathbf{X}\mathbf{X}^T$. Giả nghịch

đảo của một ma trận luôn tồn tại kể cả khi ma trận đó không vuông. Khi ma trận là vuông và khả nghịch, giả nghịch đảo chính là nghịch đảo. Tổng quát, nghiệm của bài toán tối ưu (3.5) là

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger \mathbf{X}\mathbf{y} \tag{3.9}$$

Hàm số tính giả nghịch đảo của một ma trận bất kỳ có sẵn trong thư viện numpy.

3.2.4. Hệ số điều chỉnh

Hàm dự đoán đầu ra của hồi quy tuyến tính thường có thêm một *hệ số điều chỉnh* (bias) b:

$$f(\mathbf{x}) = \mathbf{x}^T\mathbf{w} + b \tag{3.10}$$

Nếu $b = 0$, đường thẳng/mặt phẳng $\mathbf{y} = \mathbf{x}^T\mathbf{w} + b$ luôn đi qua gốc toạ độ. Việc thêm hệ số b khiến mô hình linh hoạt hơn. Hệ số điều chỉnh này cũng là một tham số mô hình.

Để ý thấy rằng, nếu coi mỗi điểm dữ liệu có thêm một đặc trưng $x_0 = 1$, ta sẽ có

$$y = \mathbf{x}^T\mathbf{w} + b = w_1x_1 + w_2x_2 + \cdots + w_dx_d + bx_0 = \bar{\mathbf{x}}^T\bar{\mathbf{w}} \tag{3.11}$$

trong đó $\bar{\mathbf{x}} = [x_0, x_1, x_2, \ldots, x_N]^T$ và $\bar{\mathbf{w}} = [b, w_1, w_2, \ldots, w_N]$. Nếu đặt $\bar{\mathbf{X}} = [\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \ldots, \bar{\mathbf{x}}_N]$, ta có nghiệm của bài toán tối thiểu hàm mất mát

$$\bar{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{2N}\|\mathbf{y} - \bar{\mathbf{X}}^T\bar{\mathbf{w}}\|_2^2 = (\bar{\mathbf{X}}\bar{\mathbf{X}}^T)^\dagger\bar{\mathbf{X}}\mathbf{y} \tag{3.12}$$

Kỹ thuật thêm một đặc trưng $x_0 = 1$ vào vector đặc trưng và ghép hệ số điều chỉnh b vào vector trọng số $\mathbf{w}$ như trên còn được gọi là *thủ thuật gộp hệ số điều chỉnh* (bias trick).

3.3. Ví dụ trên Python

3.3.1. Bài toán

Xét một ví dụ đơn giản có thể áp dụng hồi quy tuyến tính. Chúng ta sẽ so sánh nghiệm của bài toán khi giải theo phương trình (3.12) và nghiệm tìm được khi dùng thư viện scikit-learn của Python.

Giả sử có dữ liệu cân nặng và chiều cao của 15 người trong Bảng 3.1. Dữ liệu của hai người có chiều cao 155 cm và 160 cm được tách ra làm tập kiểm tra, phần còn lại tạo thành tập huấn luyện.

Bài toán đặt ra là liệu có thể dự đoán cân nặng của một người dựa vào chiều cao của họ không? Có thể thấy là cân nặng thường tỉ lệ thuận với chiều cao, vì vậy hồi quy tuyến tính là một mô hình phù hợp.

Bảng 3.1: Bảng dữ liệu về chiều cao và cân nặng của 15 người

Chiều cao (cm)	Cân nặng (kg)	Chiều cao (cm)	Cân nặng (kg)
147	49	168	60
150	50	170	72
153	51	173	63
155	52	175	64
158	54	178	66
160	56	180	67
163	58	183	68
165	59		

3.3.2. Hiển thị dữ liệu trên đồ thị

Trước tiên, ta khai báo dữ liệu huấn luyện.

```
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[147, 150, 153, 158, 163, 165, 168, 170, 173, 175, 178, 180,
    183]]).T  # height (cm), input data, each row is a data point
# weight (kg)
y = np.array([ 49, 50, 51,  54, 58, 59, 60, 62, 63, 64, 66, 67, 68])
```

Các điểm dữ liệu được minh hoạ bởi các điểm hình tròn trong Hình 3.1. Ta thấy rằng dữ liệu được sắp xếp gần như theo một đường thẳng, vậy mô hình hồi quy tuyến tính sau đây có khả năng cho kết quả tốt, với `w_0` là hệ số điều chỉnh b:

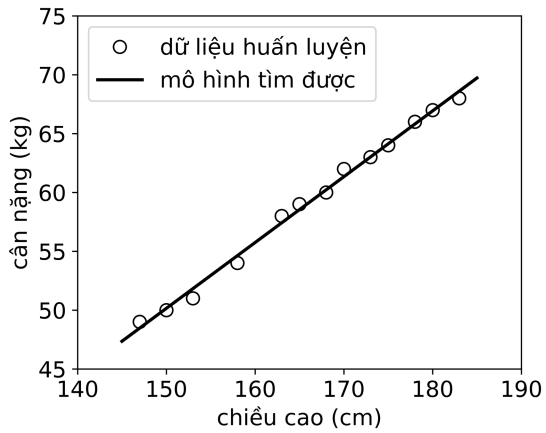
$$(\text{cân nặng}) = \texttt{w_1}*(\text{chiều cao}) + \texttt{w_0}$$

3.3.3. Nghiệm theo công thức

Tiếp theo, ta tìm các hệ số `w_1` và `w_0` dựa vào công thức (3.12). Giả nghịch đảo của một ma trận `A` trong Python được tính bằng `numpy.linalg.pinv(A)`.

```
# Building Xbar
one = np.ones((X.shape[0], 1))
Xbar = np.concatenate((one, X), axis = 1) # each row is one data point
# Calculating weights of the linear regression model
A = np.dot(Xbar.T, Xbar)
b = np.dot(Xbar.T, y)
w = np.dot(np.linalg.pinv(A), b)
# weights
w_0, w_1 = w[0], w[1]
```

Đường thẳng mô tả mối quan hệ giữa đầu vào và đầu ra được minh hoạ trong Hình 3.1. Ta thấy rằng các điểm dữ liệu nằm khá gần đường thẳng dự đoán. Vậy mô hình hồi quy tuyến tính hoạt động tốt với tập dữ liệu huấn luyện. Bây giờ, chúng ta sử dụng mô hình này để dự đoán dữ liệu trong tập kiểm tra.



Hình 3.1. Minh hoạ dữ liệu và đường thẳng xấp xỉ tìm được bởi hồi quy tuyến tính

```
y1 = w_1*155 + w_0
y2 = w_1*160 + w_0
print('Input 155cm, true output 52kg, predicted output %.2fkg.'  %(y1) )
print('Input 160cm, true output 56kg, predicted output %.2fkg.'  %(y2) )
```

Kết quả:

```
Input 155cm, true output 52kg, predicted output 52.94kg.
Input 160cm, true output 56kg, predicted output 55.74kg.
```

Chúng ta thấy rằng đầu ra dự đoán khá gần đầu ra thực sự.

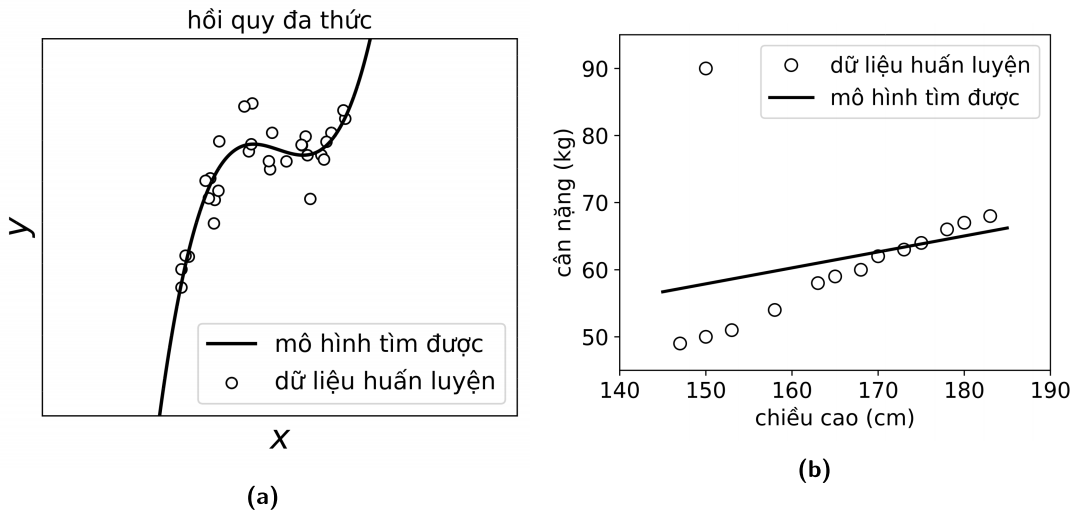
3.3.4. Nghiệm theo thư viện scikit-learn

Tiếp theo, chúng ta sẽ sử dụng thư viện scikit-learn để tìm nghiệm.

```
from sklearn import datasets, linear_model
# fit the model by Linear Regression
regr = linear_model.LinearRegression()
regr.fit(X, y) # in scikit-learn, each sample is one row
# Compare two results
print("scikit-learn's solution: w_1 = ", regr.coef_[0], "w_0 = ",\
      regr.intercept_)
print("our solution           : w_1 = ", w[1], "w_0 = ", w[0])
```

Kết quả:

```
scikit-learn solution: w_1 =  [ 0.55920496] w_0 =  [-33.73541021]
our solution         : w_1 =  [ 0.55920496] w_0 =  [-33.73541021]
```



Hình 3.2. (a) Hồi quy đa thức bậc ba (b) Hồi quy tuyến tính nhạy cảm với nhiễu.

Chúng ta thấy rằng hai kết quả thu được là như nhau.

3.4. Thảo luận

3.4.1. Hồi quy đa thức

Hàm số $y \approx f(\mathbf{x}) = \mathbf{x}^T\mathbf{w} + b$ là một hàm tuyến tính theo cả $\mathbf{w}$ và $\mathbf{x}$. Hồi quy tuyến tính có thể áp dụng cho các mô hình chỉ cần tuyến tính theo $\mathbf{w}$. Ví dụ

$$y \approx w_1x_1 + w_2x_2 + w_3x_1^2 + w_4\sin(x_2) + w_5x_1x_2 + w_0 \tag{3.13}$$

là một hàm tuyến tính theo $\mathbf{w}$ nhưng không tuyến tính theo $\mathbf{x}$. Bài toán này vẫn có thể được giải bằng hồi quy tuyến tính. Với mỗi vector đặc trưng $\mathbf{x} = [x_1, x_2]^T$, ta tính vector đặc trưng mới $\tilde{\mathbf{x}} = [x_1, x_2, x_1^2, \sin(x_2), x_1x_2]^T$ rồi áp dụng hồi quy tuyến tính với dữ liệu mới này. Tuy nhiên, việc tìm ra các hàm số $\sin(x_2)$ hay x_1x_2 là tương đối không tự nhiên. *Hồi quy đa thức* thường được sử dụng nhiều hơn với các vector đặc trưng mới có dạng $[1, x_1, x_1^2, \dots]^T$ (đặc trưng đa thức). Một ví dụ về hồi quy đa thức bậc 3 được thể hiện trong Hình 3.2a.

3.4.2. Hạn chế của hồi quy tuyến tính

Hạn chế đầu tiên của hồi quy tuyến tính là nó rất *nhạy cảm với nhiễu*. Trong ví dụ về mối quan hệ giữa chiều cao và cân nặng bên trên, nếu có chỉ một cặp dữ liệu nhiễu (150 cm, 90kg) thì kết quả sẽ sai khác đi rất nhiều (xem Hình 3.2b).

Một kỹ thuật giúp tránh hiện tượng này là loại bỏ các nhiễu trong quá trình tìm nghiệm. Việc làm này có thể phức tạp và tương đối tốn thời gian. Có một cách khác giúp tránh công việc loại bỏ nhiễu là sử dụng *mất mát Huber*. Hồi quy tuyến

tính với mất mát Huber được gọi là *hồi quy Huber*, được khẳng định là có khả năng kháng nhiễu tốt hơn.

Hạn chế thứ hai của hồi quy tuyến tính là nó *không biểu diễn được các mô hình phức tạp*. Mặc dù phương pháp này có thể được áp dụng nếu quan hệ giữa đầu ra và đầu vào là phi tuyến, mối quan hệ này vẫn đơn giản hơn nhiều so với các mô hình thực tế. Hơn nữa, việc tìm ra các đặc trưng $x_1^2, \sin(x_2), x_1x_2$ như trên là không khả thi khi số chiều dữ liệu lớn lên.

3.4.3. Điều chuẩn

Có một kỹ thuật nhỏ giúp tránh trường hợp $\mathbf{X}\mathbf{X}^T$ không khả nghịch là biến nó thành $\mathbf{A} = \mathbf{X}\mathbf{X}^T + \lambda\mathbf{I}$ với λ là một số dương nhỏ và $\mathbf{I}$ là ma trận đơn vị với bậc phù hợp.

Ma trận $\mathbf{A}$ là khả nghịch vì nó là một ma trận xác định dương. Thật vậy, với mọi $\mathbf{w} \neq \mathbf{0}$,

$$\mathbf{w}^T\mathbf{A}\mathbf{w} = \mathbf{w}^T(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})\mathbf{w} = \mathbf{w}^T\mathbf{X}\mathbf{X}^T\mathbf{w} + \lambda\mathbf{w}^T\mathbf{w} = \|\mathbf{X}^T\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2 > 0. \tag{3.14}$$

Lúc này, nghiệm của bài toán là $\mathbf{y} = (\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$.

Xét hàm mất mát

$$\mathcal{L}_2(\mathbf{w}) = \frac{1}{2N}(\|\mathbf{y} - \mathbf{X}^T\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2). \tag{3.15}$$

Phương trình gradient theo $\mathbf{w}$ bằng không:

$$\frac{\nabla\mathcal{L}_2(\mathbf{w})}{\nabla\mathbf{w}} = \mathbf{0} \Leftrightarrow \frac{1}{N}(\mathbf{X}(\mathbf{X}^T\mathbf{w} - \mathbf{y}) + \lambda\mathbf{w}) = \mathbf{0} \Leftrightarrow (\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}\mathbf{y} \tag{3.16}$$

Ta thấy $\mathbf{w} = (\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$ chính là nghiệm của bài toán tối thiểu $\mathcal{L}_2(\mathbf{w})$ trong (3.15). Mô hình học máy với hàm mất mát (3.15) còn được gọi là *hồi quy ridge*. Ngoài việc giúp phương trình gradient theo hệ số bằng không có nghiệm duy nhất, hồi quy ridge còn giúp mô hình tránh được quá khớp. Ngoài ra còn có các hồi quy dạng điều chuẩn khác như hồi quy Lasso, hồi quy ElasticNet.

3.4.4. Phương pháp tối ưu khác

Hồi quy tuyến tính là một mô hình đơn giản, lời giải cho phương trình đạo hàm bằng không cũng không phức tạp. Trong hầu hết các trường hợp, việc giải các phương trình gradient bằng không tương đối phức tạp. Tuy nhiên, nếu ta tính được đạo hàm của hàm mất mát, các tham số mô hình có thể được giải bằng một phương pháp hữu dụng có tên *Hạ gradient - gradient descent (GD)*. Trên thực tế, một vector đặc trưng có thể có kích thước rất lớn, dẫn đến ma trận $\mathbf{X}\mathbf{X}^T$ cũng có kích thước lớn và việc tính ma trận nghịch đảo có thể không lợi về mặt tính

toán. Gradient descent sẽ giúp tránh được việc tính ma trận nghịch đảo. Thuật toán GD tối ưu hàm mất mát $\mathcal{L}(\mathbf{w})$ bắt đầu bằng một điểm dự đoán $\mathbf{w}_0$, sau đó sử dụng quy tắc cập nhật:

$$\boxed{\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)} \tag{3.17}$$

Chi tiết về thuật toán GD sẽ được trình bày trong bài 6.

3.5. Câu hỏi ôn tập bài 3

Câu 1: Mô hình hồi quy tuyến tính có bias và không có bias khác nhau như thế nào.

Câu 2: Viết hàm mất mát của hồi quy tuyến tính sử dụng sai số MSE.

Câu 3: Viết đạo hàm hàm mất mát, giải phương trình đạo hàm bằng 0 tìm nghiệm tối ưu.

Câu 4: Viết công thức lặp theo GD tìm nghiệm tối ưu của hồi quy tuyến tính.

Câu 5: Điều chuẩn là gì. Mục đích của điều chuẩn là gì. Các mô hình điều chuẩn cụ thể trong hồi quy tuyến tính.

Câu 6: Hồi quy đa thức xử lý đặc trưng như thế nào để đưa về hồi quy tuyến tính? Lượng biến phụ sinh ra tính theo công thức nào?

Bài 4

K lân cận

Mục đích

Học xong bài này, sinh viên cần hiểu rõ và áp dụng được:

- *KNN hoạt động như thế nào?*
- *Bài toán phân lớp với KNN.*
- *Bài toán hồi quy với KNN*
- *Các loại trọng số sử dụng trong việc lấy giá trị, biểu quyết KNN.*
- *Ưu nhược điểm của KNN.*

4.1. Giới thiệu

4.1.1. K lân cận

K lân cận - KNN là một trong những thuật toán học có giám sát đơn giản nhất. Khi huấn luyện, thuật toán này gần như *không học* một điều gì từ dữ liệu huấn luyện mà *ghi nhớ* lại một cách máy móc toàn bộ dữ liệu đó. Mọi tính toán được thực hiện tại pha kiểm tra. KNN có thể được áp dụng vào các bài toán phân loại và hồi quy. KNN còn được gọi là một thuật toán *lười học*.

KNN là thuật toán đi tìm đầu ra của một điểm dữ liệu mới chỉ dựa trên thông tin của K điểm dữ liệu gần nhất trong tập huấn luyện.

Với KNN, mọi điểm trong tập huấn luyện đều được mô hình mô tả một cách chính xác. Việc này khiến quá khớp dễ xảy ra với KNN.

Mặc dù có nhiều hạn chế, KNN vẫn là giải pháp đầu tiên nên nghĩ tới khi giải quyết một bài toán học máy. Khi làm các bài toán học máy nói chung, không có

mô hình đúng hay sai, chỉ có mô hình cho kết quả tốt hơn. Chúng ta luôn cần một mô hình đơn giản để giải quyết bài toán, sau đó mới dần tìm cách tăng chất lượng của mô hình.

4.2. Phân tích toán học

Không có hàm mất mát hay bài toán tối ưu nào cần thực hiện trong quá trình huấn luyện KNN. Mọi tính toán được tiến hành ở bước kiểm tra. Vì KNN ra quyết định dựa trên các điểm gần nhất nên có hai vấn đề ta cần lưu tâm. Thứ nhất, khoảng cách được định nghĩa như thế nào. Thứ hai, cần phải tính toán khoảng cách như thế nào cho hiệu quả.

Với vấn đề thứ nhất, mỗi điểm dữ liệu được thể hiện bằng một vector đặc trưng, khoảng cách giữa hai điểm chính là khoảng cách giữa hai vector đó. Có nhiều loại khoảng cách khác nhau tuỳ vào bài toán, nhưng khoảng cách được sử dụng nhiều nhất là khoảng cách Euclid.

Vấn đề thứ hai cần được lưu tâm hơn, đặc biệt với các bài toán có tập huấn luyện lớn và vector dữ liệu có kích thước lớn. Giả sử các vector huấn luyện là các cột của ma trận $\mathbf{X} \in \mathbb{R}^{d \times N}$ với d và N lớn. KNN sẽ phải tính khoảng cách từ một điểm dữ liệu mới $\mathbf{z} \in \mathbb{R}^d$ đến toàn bộ N điểm dữ liệu đã cho và chọn ra K khoảng cách nhỏ nhất. Nếu không có cách tính hiệu quả, khối lượng tính toán sẽ rất lớn.

Tiếp theo, chúng ta cùng thực hiện một vài phân tích toán học để tính các khoảng cách một cách hiệu quả. Ở đây khoảng cách được xem xét là khoảng cách Euclid.

Khoảng cách từ một điểm tới từng điểm trong một tập hợp

Khoảng cách Euclid từ một điểm $\mathbf{z}$ tới một điểm $\mathbf{x}_i$ trong tập huấn luyện được định nghĩa bởi $\|\mathbf{z}-\mathbf{x}_i\|_2$. Người ta thường dùng bình phương khoảng cách Euclid $\|\mathbf{z}-\mathbf{x}_i\|_2^2$ để tránh phép tính căn bậc hai. Việc bình phương này không ảnh hưởng tới thứ tự của các khoảng cách. Để ý rằng

$$\|\mathbf{z}-\mathbf{x}_i\|_2^2 = (\mathbf{z}-\mathbf{x}_i)^T(\mathbf{z}-\mathbf{x}_i) = \|\mathbf{z}\|_2^2 + \|\mathbf{x}_i\|_2^2 - 2\mathbf{x}_i^T\mathbf{z} \tag{4.1}$$

Để tìm ra $\mathbf{x}_i$ gần với $\mathbf{z}$ nhất, số hạng đầu tiên có thể được bỏ qua. Hơn nữa, nếu có nhiều điểm dữ liệu trong tập kiểm tra, các giá trị $\|\mathbf{x}_i\|_2^2$ có thể được tính và lưu trước vào bộ nhớ. Khi đó, ta chỉ cần tính các tích vô hướng $\mathbf{x}_i^T\mathbf{z}$.

Khoảng cách giữa từng cặp điểm trong hai tập hợp

Thông thường, tập kiểm tra bao gồm nhiều điểm dữ liệu tạo thành một ma trận $\mathbf{Z}$. Ta phải tính từng cặp khoảng cách giữa mỗi điểm trong tập kiểm tra và một điểm trong tập huấn luyện. Nếu mỗi tập có 1000 phần tử, có một triệu khoảng cách cần tính. Nếu không có phương pháp tính hiệu quả, thời gian thực hiện sẽ rất dài.